# Spark 读数据与清洗交付说明

## 1. 本阶段完成内容

本阶段已完成大数据平台技术项目中 **Spark 读取数据与数据清洗** 部分。

本次处理的数据来源为 HDFS 中的项目数据目录：

```
hdfs://master:9000/aiops/metric
```

该目录下包含 6 个实验场景数据：

```
2022-03-20-cloudbed1
2022-03-20-cloudbed2
2022-03-20-cloudbed3
2022-03-21-cloudbed1
2022-03-21-cloudbed2
2022-03-21-cloudbed3
```

每个场景中包含 5 类监控指标数据：

```
service
node
container
jvm
istio
```

本阶段使用 Spark 对 HDFS 中的原始 CSV 数据进行了读取、类型转换、缺失值过滤和统一格式整理，并将清洗后的结果重新写回 HDFS，供后续异常检测算法使用。

## 2. 运行方式

由于服务器环境中没有可用的 Python3，且 YARN 模式暂时存在资源调度问题，因此最终采用 Spark Scala 脚本，并使用 `local[*]` 模式完成计算。

虽然没有走 YARN 分布式调度，但本次任务仍然使用 Spark 读取 HDFS 数据，并将处理结果写回 HDFS。

执行命令如下：

```
spark-shell --master local[*] -i /root/spark_read_clean.scala
```

## 3. 清洗逻辑

本阶段主要完成以下清洗工作：

1. 递归读取 `/aiops/metric` 下 6 个场景中的监控 CSV 文件。
2. 分别读取 `service`、`node`、`container`、`jvm`、`istio` 五类数据。
3. 将 `timestamp` 字段转换为 `long` 类型。
4. 将 `rr`、`sr`、`count`、`mrt`、`value` 等指标值转换为 `double` 类型。
5. 删除 `timestamp`、对象 ID、指标名、指标值为空或格式非法的记录。
6. 将 `service` 类业务指标从宽表转换为长表。
7. 将所有指标统一整理为时间序列分析所需的标准格式。

清洗后的统一字段结构如下：

```
timestamp, object_id, kpi_name, value, source_file, metric_type
```

字段含义如下：

| 字段 | 含义 |
| --- | --- |
| `timestamp` | 采样时间戳 |
| `object_id` | 服务、节点、容器、JVM 或 Istio 对象 |
| `kpi_name` | 指标名称 |
| `value` | 指标值 |
| `source_file` | 数据来源文件 |
| `metric_type` | 指标类型，包括 service、node、container、jvm、istio |

## 4. 输出位置

清洗后的结果已写入 HDFS：

```
hdfs://master:9000/aiops/output/clean_metrics
```

可以通过以下命令查看输出目录：

```
hdfs dfs -ls /aiops/output/clean_metrics
```

正常情况下应包含 `_SUCCESS` 文件，以及按指标类型分区的目录：

```
metric_type=container
metric_type=istio
metric_type=jvm
```

```
metric_type=node
metric_type=service
```

## 5. 清洗结果统计

本次 Spark 清洗任务最终输出统计如下：

```
清洗后总记录数：44,972,147
构造出的时间序列数量：8,730
```

各类指标数量如下：

| 指标类型 | 记录数 |
|---|---|
| container | 22,211,676 |
| istio | 13,695,910 |
| jvm | 5,750,910 |
| node | 2,933,643 |
| service | 380,008 |

该结果说明项目数据已成功被 Spark 读取并完成统一清洗。数据中实际构造出 8,730 条时间序列，满足项目中 5000+ 时间序列的要求。

## 6. 后续使用方式

后续异常检测阶段可以直接读取清洗后的 Parquet 数据。

Scala 读取方式：

```
val df =
spark.read.parquet("hdfs://master:9000/aiops/output/clean_metrics")
```

PySpark 读取方式：

```
df = spark.read.parquet("hdfs://master:9000/aiops/output/clean_metrics")
```

后续可以基于 `(object_id, kpi_name)` 分组构造时间序列，并实现 IQR、Z-score 等异常检测算法。

## 7. 当前阶段结论

本阶段已完成从 HDFS 读取原始监控数据、Spark 清洗、统一时间序列格式构造和清洗结果写回 HDFS。

该阶段产出的 `clean_metrics` 数据集可作为后续异常检测、根因分析和可视化展示的基础数据。